

VOLUME 12 / RUST BACKEND MASTERY

Fly.io 公開 — Docker・CI・運用・復旧

本番用の実行ファイルを作るところから、公開・見守り・元に戻すところまでを、手を動かして確かめる

第12巻: Fly.io 公開 — Docker・CI・運用・復旧

この巻で身に付けること

第11巻までで、RustNote は自分の PC の中で、正しく動くようになりました。この巻では、それを**インターネットに公開し、公開したあとも安全に動かし続ける**方法を学びます。

自分の PC で動くことと、公開して動き続けることの間には、思っているより大きな差があります。公開してから、次のような事故が起きることがよくあります。

- 自分の PC では速かったのに、公開したらログインに何秒もかかる（試し用の作り方のまま公開していた）
- 公開した画面の CSS が読み込まれず、文字だけの画面になる（実行ファイルだけを置いて、見た目のファイルを置き忘れた）
- 「どの版のコードが動いているのか」が、誰にも分からない
- 秘密のトークンを、設定ファイルに書いたまま共有してしまう
- 新しい版を出したら起動しなくなり、前の版に戻す方法が分からない
- DB のデータが消えたのに、元に戻す手順を一度も試していなかった

この巻のテーマは、こうした事故を、**作る手順・置く場所・確かめる方法・戻す方法**を先に決めておくことで防ぐことです。

到達目標

この巻を終えると、次のことができるようになります。

- 本番用の実行ファイル（release ビルド）と、試し用（debug ビルド）の違いを、速さ・大きさ・動きの違いで説明できる。どのコミットと、どの Rust から作った実行ファイルかを記録できる
- multi-stage の Dockerfile で、作る場所と動かす場所を分けた小さなコンテナイメージを作れる
- Fly.io の設定（`fly.toml`）と、アプリが待ち受けるアドレス・ポートをそろえ、秘密の値を `fly secrets` で渡せる
- ヘルスチェックと起動の順番、migration を出す順番を決め、公開したあとにログと動作確認で「本当に動いているか」を確かめられる
- 台数・地域・費用の決め方と、前の版に戻す・DB を元に戻すといった障害のときの手順を、あらかじめ試しておける

学習地図

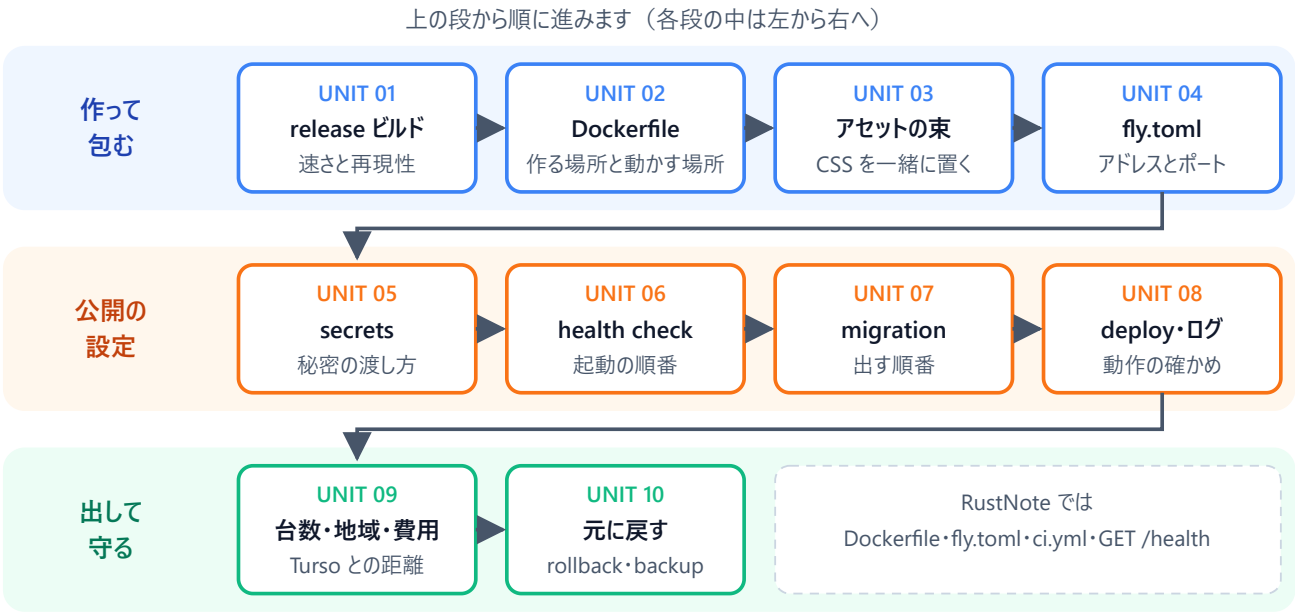


図12-1: 第12巻の学習地図。本番用の実行ファイルを作って包み、公開の設定を決め、出したあとに見守って、困ったら元に戻す

単元	テーマ	ひとことで言うと
UNIT 01	release ビルドと再現性	本番用の実行ファイルを作り、同じソースから同じ物ができるかを確かめる
UNIT 02	multi-stage の Dockerfile	作る場所と動かす場所を分け、小さく安全なイメージにする
UNIT 03	アセットの束を本番用に作る（準備中）	`topcoat asset bundle --release` と、`assets/` の置き場所
UNIT 04	fly launch・fly.toml・ポート（準備中）	`0.0.0.0:8080` と `internal_port` をそろえる
UNIT 05	secrets・設定・入れ替え（準備中）	トークンを、イメージにも `fly.toml` にも書かない
UNIT 06	health check と起動の順番（準備中）	「動いている」と「受け付けてよい」を外から確かめる
UNIT 07	migration を出す順番と release command（準備中）	新しい表と古いコードがぶつからないようにする
UNIT 08	deploy・ログ・動作の確かめ・独自ドメイン（準備中）	出したあとに、本当に動いているかを確かめる
UNIT 09	台数・地域・費用・Turso との距離（準備中）	速さと費用の釣り合いを、数で決める
UNIT 10	rollback・backup・障害からの復旧・更新（準備中）	前の版と前のデータに戻す手順を、先に試しておく

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03〜10 は準備中で、次の更新から順に追加します。この巻では、Fly.io と Turso のアカウントを使った実際の公開はしません。アカウントが必要な操作（ `fly launch`・`fly deploy`・`fly secrets`・`turso db create` など）は、コマンドと、公式のドキュメントで確かめた手順の説明だけを載せ、実行結果は載せません。実行結果を載せるのは、自分の PC で実際に動かせるもの（release ビルド、Docker でのビルドと起動、ローカルの DB ファイルなど）だけです。

前提知識（先に読んでおく巻）

- **第1巻（環境構築）**：PowerShell・Git・Rust が使えること。この巻の後半では、Docker・flyctl（Fly.io のコマンド）・Turso CLI も使います（それぞれの単元で、入れ方から説明します）。
- **第5巻（async）**：`spawn_blocking` で重い計算を別のスレッドに任せること（第5巻 UNIT 07）。UNIT 01 で、パスワードのハッシュの速さを測ります。
- **第9巻（RustNote 設計）**：RustNote のフォルダ構成と、設定・起動の順番。

- **第13巻（完全コード）UNIT 01:** `Cargo.toml`・`Cargo.lock`・`rust-toolchain.toml` と `--locked`。この巻の UNIT 01 は、その続きとして読めます。

> **注意:** この巻の本文は、教材のフォルダの `Rust/rustnote/` のコードと、この巻で実際に動かした結果だけをもとに書いています。RustNote の改善点（第11巻・第13巻で見つけたもの）は、教材のコードでは直さずにそのまま使い、関係する所で、その都度ふれます。

この巻の進め方

公開の作業は、一度うまくいくと、手順を忘れてしまいがちです。各単元では、次の順番で進めてください。

1. **予想する:** コマンドを実行する前に、何ができあがり、何が表示されるかを予想します。
2. **そのまま動かす:** 教材のコマンドをそのまま実行し、結果を比べます。
3. **記録する:** 何を、どの版で、いつ作ったかをメモします（UNIT 01 の問3で、記録の作り方を練習します）。
4. **元に戻す:** 試しに変えた設定は、元に戻してから次へ進みます。公開した物を元に戻す方法は、UNIT 10 で学びます。

> **ヒント:** この巻の実行結果には、かかった時間（`1m 25s` や `20.25ms` など）や、ファイルの指紋（SHA-256 の長い英数字）が出てきます。時間は PC の速さで、指紋は作った PC と時刻で変わります。何がなぜ変わるのかは、UNIT 01 で確かめます。

使うツールと版

この巻のコードは、すべて次の環境で実際にビルド・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.99 以上**（Topcoat 0.11 の必要条件）。動作確認は rustc 1.99.1 / cargo 1.99.1（`aarch64-pc-windows-msvc`）
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では `src/` を変えずに使う
topcoat-cli	0.11.0（`topcoat asset bundle` で CSS の束を作る。RustNote の topcoat と同じ版）
Docker	29.3.1（Docker Desktop。中は linux/arm64。UNIT 02 から使います。イメージは `rust:1.99.1-trixie` と `debian:trixie-slim`）
flyctl	v0.5.1（Fly.io のコマンド。Windows 用の公式の配布は x86_64 版だけで、この PC（ARM64）でも `windows/amd64` 版が動きます。UNIT 04 から使います。アカウントへのログインはしません）

練習用のフォルダを用意する

第9巻・第13巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。この巻では、確かめ用の小さなプログラムを `examples/` に足していきます。RustNote の `src/` は変えません。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる（1.99.1 になる）
rustup show active-toolchain
```

topcoat CLI がまだ入っていなければ、ここで入れておきます。RustNote の `Dockerfile` と同じ版・同じ書き方です（ダウンロードとコンパイルをするので、数分かかることがあります）。

```
cargo install topcoat-cli --version 0.11.0 --locked
topcoat --version
```

```
topcoat 0.11.0
```

UNIT 01: release ビルドと再現性

この単元のゴール

- debug ビルド（試し用）と release ビルド（本番用）の違いを、速さ・大きさ・整数があふれたときの動きの3つで、実際の数を使って説明できる
- 本番用の実行ファイルを作る手順（`cargo build --release --locked` → `topcoat asset bundle --release`）を説明し、手元で起動して `GET /health` を確かめられる
- 同じソースから2回作った実行ファイルが同じ中身になるか（再現性）を確かめ、違いの原因（リンクした時刻・フォルダの場所）を説明できる
- どのコミット・どの Rust・どの `Cargo.lock` から作った実行ファイルかを、記録として残せる

たとえ話でつかむ

パン屋さんが、新しいパンを売り出すまでを考えます。

- **試作**では、生地をこねるたびに手を止めて、温度や重さを確かめます。焼き上がるまで時間がかかりますが、何かおかしければ、すぐにその場で気づけます。
- **本番の仕込み**では、確かめの手順を減らして、速くたくさん焼きます。そのかわり、試作のときなら止まって気づけた小さな間違いが、そのまま店に並ぶことがあります。

- 焼いたパンだけでは、売れません。**値札と袋**（見た目）を、パンと一緒に棚に並べる必要があります。
- 袋には、**製造記録**（いつ・どのレシピで・どの小麦粉の袋から作ったか）を残します。お客さんから「昨日のパンがおかしい」と言われたとき、どの材料で作ったかをたどれるようにするためです。
- 同じレシピ・同じ材料で焼けば、同じパンになります。ただし、袋に押す**日付の印**だけは、毎回違います。

RustNote の公開の準備も、同じ形をしています。

パン屋さん	RustNote
試作（確かめながら、ゆっくり）	**debug ビルド**（`cargo build`。あふれの確かめ付き・最適化なし）
本番の仕込み（速く、たくさん）	**release ビルド**（`cargo build --release`。最適化あり・あふれの確かめなし）
値札と袋	**アセットの束**（`topcoat asset bundle --release` が作る `assets/`）
製造記録	**ビルドの記録**（コミット・Rust の版・`Cargo.lock` と実行ファイルの指紋）
袋の日付の印	実行ファイルに入る**リンクした時刻**

> **用語:** debug ビルド（デバッグビルド： `cargo build` で作る、試し用の実行ファイル。コンパイルが速く、間違いに気づきやすいが、動きは遅い。`target\debug` にできる）

> **用語:** release ビルド（リリースビルド： `cargo build --release` で作る、本番用の実行ファイル。コンパイルに時間をかけて最適化し、速く動く。`target\release` にできる）

> **用語:** 再現性（さいげんせい：同じソース・同じ道具・同じ部品から作れば、いつ誰が作っても同じ物ができること。英語では reproducibility）

図で理解する

同じ RustNote のソースから作った2つの実行ファイルを、この PC で比べると、次のようになりました（数は、この単元の例で実際に測ったものです）。

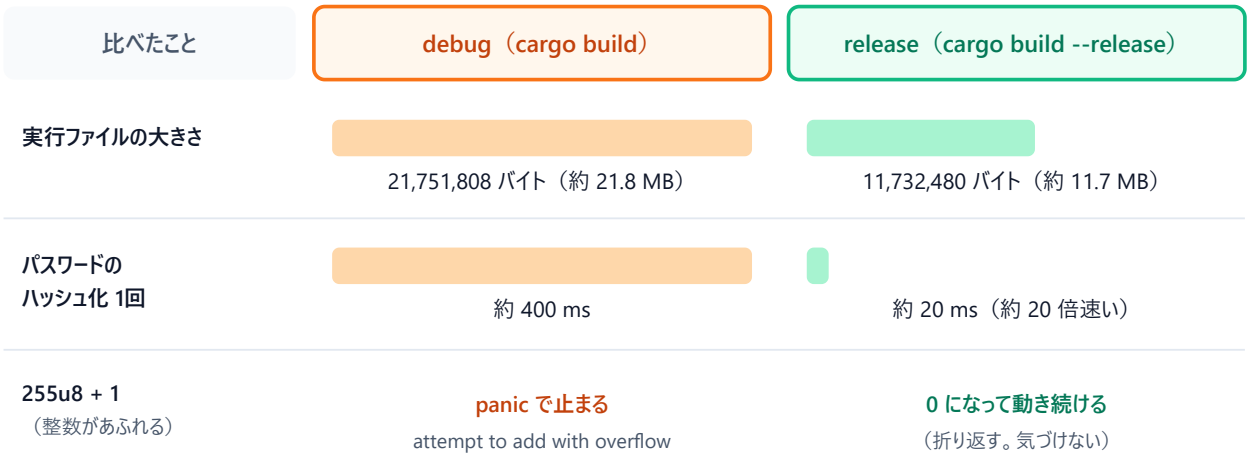


図12-2: 同じソースから作った debug と release の違い。release は小さく速いが、整数のあふれを確かめない。どの数も、この PC（Windows ARM64）で実際に測ったもの

- **大きさ:** release は、debug の約半分でした。最適化で、使われない部分や、確かめのためのコードが減るからです。
- **速さ:** パスワードのハッシュ化（Argon2id）は、release のほうが約 20 倍速くなりました。ログインのたびに行う計算なので、この差は、利用者が待つ時間にそのまま出ます。
- **あふれ:** debug では、整数があふれると panic で止まります。release では、何も言わずに折り返して（`255 + 1` が `0` になって）動き続けます。release のほうが、間違いに気づきにくいのです（エラー3）。

本番用の実行ファイルは、次の順番で作ります。RustNote の `Dockerfile` も、同じ順番で作っています（UNIT 02（準備中））。

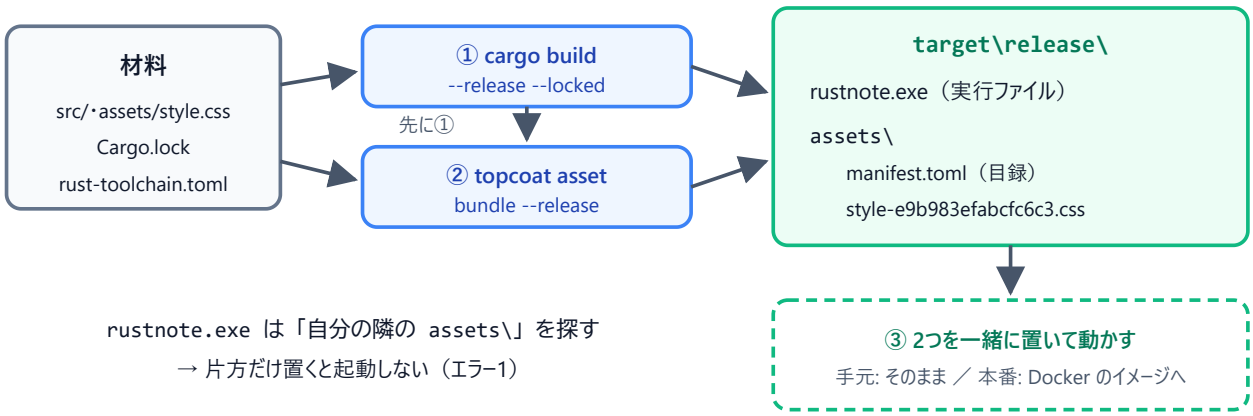


図12-3: 本番用の実行ファイルができるまで。① release ビルドの後に ② アセットの束を作り、2つを必ず一緒に置く

- **① release ビルド**で、`target\release\rustnote.exe` ができます。`--locked` で、`Cargo.lock` のとおりの版を使います（第13巻 UNIT 01）。

- ② **アセットの束**は、`topcoat asset bundle --release` で作ります。topcoat CLI は、中で `cargo build --release` を動かして実行ファイルを用意し（①で作ってあれば、そのまま使います）、その中から `asset!("assets/style.css")` の印を探して、CSS を `style-e9b983efabcfc6c3.css` のような名前（中身から計算した指紋付き）にして、実行ファイルの隣の `assets\` に置きます。topcoat CLI の中の `cargo build` には `--locked` が付かないので、先に①で、`Cargo.lock` のとおりの版で作っておきます（くわしくは UNIT 03（準備中））。
- ③ 実行ファイルは、起動するとき「**自分の隣の** `assets\` 」を読みます。片方だけをコピーすると、起動しません。

まず動かす最小コード

例1-1: release ビルドを作り、起動して確かめる

> **▼ このコマンドがやること（先に日本語で）：** `Cargo.lock` のとおりの版で release ビルドを作り、CSS の束を作ってから、できた実行ファイルを起動します。別の PowerShell の窓から `GET /health` を送り、`{"status":"ok"}` が返ることを確かめます。

```
# ① 本番用にビルドする（Cargo.lock を書き換えない）
cargo build --release --locked
# ② 実行ファイルの隣に、CSS の束を作る
topcoat asset bundle --release
# ③ 起動する（止めるときは Ctrl+C）
.\target\release\rustnote.exe
```

▼ 実行結果（`cargo build` は `Compiling` の行が 289 行続くので、最後の2行だけを載せています。クレートをまだダウンロードしていない PC では、その表示も加わります）

```
   Compiling rustnote v0.1.0 (...\rustnote)
    Finished `release` profile [optimized] target(s) in 1m 26s
```

```
bundled assets into target\release\assets
```

```
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://127.0.0.1:3000
```

起動したまま、もう1つの PowerShell の窓で、ヘルスチェックの URL を開きます。`curl.exe` の `-i` は、本文の前に、返事の見出し（ヘッダー）も表示する指定です。

```
curl.exe -i http://127.0.0.1:3000/health
```

```
HTTP/1.1 200 OK
content-type: application/json
content-length: 15
date: Fri, 30 Oct 2026 22:19:10 GMT

{"status":"ok"}
```

▼ コードを1つずつ分解して解説

解説1: release は `target\release` にできる

- `Finished release profile [optimized]` の `optimized` は「最適化した」という意味です。debug ビルドのときは `Finished dev profile [unoptimized + debuginfo]`（最適化なし・デバッグ用の情報付き）と表示されます。
- debug ビルドは `target\debug\`、release ビルドは `target\release\` と、別のフォルダにできます。2つは混ざりません。この PC では、何もない状態からの release ビルドに 1 分 26 秒かかりました。debug ビルドは 1 分 6 秒でした。release は、最適化に時間を使うぶん遅くなります。

解説2: bundle は「実行ファイルの隣」に作る

- `bundled assets into target\release\assets` は、「`target\release\assets` に束を作りました」です。`--release` を付けたので、release の実行ファイルの隣にできました。付け忘れると `target\debug\assets` にできてしまいます（エラー1）。
- 実際の表示では、`target` の前に、フォルダの場所が付きます。

解説3: 起動の表示とヘルスチェック

- 新しい DB ファイル（`rustnote.db`）だったので、起動のときに3つのマイグレーションが適用されました。2回目からは、この3行は出ません。DB ファイルは、コマンドを実行したフォルダ（今いるフォルダ）にできます。`.env` が無いときの `DATABASE_URL` の既定値が `rustnote.db` だからです（第13巻 UNIT 01）。
- `GET /health` は 200 で `{"status":"ok"}` を返しました。RustNote の `/health` は、DB に `SELECT 1` を送って、答えが返れば `ok` です。Fly.io も、公開したあと、この URL で「動いているか」を確かめます（UNIT 06（準備中））。

例1-2: debug と release の速さを比べる

> **▼ このコードがやること（先に日本語で）**: RustNote の `auth::hash_password`（パスワードを Argon2id でハッシュ化する）と `auth::verify_password`（照合する）を3回ずつ呼び、それぞれにかかった時間を表示します。同じプログラムを、debug と release の2通りで動かして比べます。

```rust
//! RustNote のパスワードのハッシュ化と照合（Argon2id）にかかる時間を、3回ずつ測る。
//! debug と release で比べる:
//!   cargo run -q --example u01_hash_speed
//!   cargo run -q --release --example u01_hash_speed

use std::time::Instant;

use rustnote::auth::{hash_password, verify_password};
use rustnote::domain::Password;

#[tokio::main]
async fn main() -> anyhow::Result<()> {
    // debug ビルドでは debug_assertions が有効、release ビルドでは無効になる
    let profile = if cfg!(debug_assertions) {
        "debug"
    } else {
        "release"
    };
    println!("ビルドの種類: {profile}");

    let password = Password::parse("correct horse battery")?;
    for n in 1..=3 {
        let started = Instant::now();
        let phc = hash_password(&password).await?;
        let hashed = started.elapsed();

        let started = Instant::now();
        let matches = verify_password("correct horse battery", &phc).await?;
        let verified = started.elapsed();

        println!("{n}回目: ハッシュ化 {hashed:.2?}・照合 {verified:.2?}（一致:
{matches}）");
    }
    Ok(())
}
```

`examples/u01_hash_speed.rs` として保存し、`--release` を付けずに1回、付けて1回実行します。

```
cargo run -q --example u01_hash_speed
cargo run -q --release --example u01_hash_speed
```

▼ 実行結果

```
ビルドの種類: debug
1回目: ハッシュ化 401.18ms・照合 445.38ms（一致: true）
2回目: ハッシュ化 409.09ms・照合 412.93ms（一致: true）
3回目: ハッシュ化 396.51ms・照合 409.86ms（一致: true）
```

```
ビルドの種類: release
1回目: ハッシュ化 19.66ms・照合 24.79ms（一致: true）
2回目: ハッシュ化 20.01ms・照合 20.61ms（一致: true）
3回目: ハッシュ化 21.73ms・照合 19.39ms（一致: true）
```

▼ コードを1つずつ分解して解説

解説1: cfg!(debug_assertions) で、どちらのビルドかが分かる

- `cfg!(debug_assertions)` は、debug ビルドでは `true`、release ビルドでは `false` になります。`debug_assert!`（debug のときだけ確かめる `assert!`）は、この設定で有効・無効が切り替わります。整数のあふれの確かめも、既定では debug で有効・release で無効です（エラー3）。

解説2: 約 20 倍の差は、ログインの待ち時間の差

- ハッシュ化は、debug で約 400 ms、release で約 20 ms でした。Argon2id は、わざと計算を重くして、パスワードを総当たりで試されにくくする仕組みです（第8巻）。重い計算なので、最適化の効き目がとても大きく出ます。
- debug のままの実行ファイルを公開すると、ログインと登録のたびに、利用者を 0.4 秒近く余計に待たせることになります。しかも、そのあいだ CPU を使い続けるので、同時にログインする人が増えると、もっと遅くなります。**公開するのは release ビルド**です。

解説3: 1回目だけ少し遅い

- debug の1回目の照合（445.38 ms）は、2回目以降（410 ms ほど）より少し遅くなりました。最初の1回は、計算に使うメモリを用意したり、`spawn_blocking` 用のスレッドを作ったりする分が加わることがあるからだと考えられます。ただし、ハッシュ化のほうは、1回目と2回目以降でほとんど差がありませんでした。速さを比べるときは、1回だけで決めず、何回か測ります。

もう一歩: 実務に近い形

公開した実行ファイルについて、「これは、どのソースから作った物か」を後から確かめられることが大切です。同じソースから作れば、いつでも同じ実行ファイルになるのでしょうか。確かめるための小さな道具を作ります。

例1-3: 実行ファイルを調べる道具を作る

> **▼ このコードがやること（先に日本語で）**：渡された実行ファイル（.exe）を読み、大きさ、リンクした時刻（実行ファイルの見出しに書かれている）、ユーザーフォルダの場所（ `C:\Users\あなたの名前` の部分）が中に何か所入っているかを表示します。2つ渡すと、中身が何バイト違うかも数えます。外のクレートは使わず、標準ライブラリだけで書きます。

```rust
//! Windows の実行ファイル（.exe）を調べて、大きさ・リンクした時刻・ユーザーフォルダの
//! 場所が何か所入っているかを表示する。2つ渡すと、違うバイトの数も数える。
//! 実行: cargo run -q --example u01_exe_report -- 1つ目.exe [2つ目.exe]

use std::env;
use std::fs;

/// PE 形式（Windows の実行ファイルの形式）の見出しから、リンクした時刻を読む。
/// 時刻は「1970-01-01 からの秒数」（UNIX 時刻）で入っている
fn link_time(bytes: &[u8]) -> Option<u32> {
    // 先頭から 0x3C の位置に、PE の見出しがどこから始まるかが書いてある
    let header: [u8; 4] = bytes.get(0x3C..0x40)?.try_into().ok()?;
    let pe = usize::try_from(u32::from_le_bytes(header)).ok()?;
    // 見出しは "PE\0\0" で始まり、その 8 バイト後ろに時刻がある
    if bytes.get(pe..pe + 4)? != b"PE\0\0" {
        return None;
    }
    let time: [u8; 4] = bytes.get(pe + 8..pe + 12)?.try_into().ok()?;
    Some(u32::from_le_bytes(time))
}

/// `needle` が何か所に入っているかを数える
fn count(haystack: &[u8], needle: &[u8]) -> usize {
    haystack
        .windows(needle.len())
        .filter(|window| *window == needle)
        .count()
}

fn main() -> std::io::Result<()> {
    let paths: Vec<String> = env::args().skip(1).collect();
    // ユーザーフォルダの場所（例: C:\Users\taro）。場所そのものは表示しない
    let profile = env::var("USERPROFILE").unwrap_or_default();

    let mut files = Vec::new();
    for path in &paths {
        let bytes = fs::read(path)?;
        println!("[{path}]");
        println!("    大きさ: {} バイト", bytes.len());
        match link_time(&bytes) {
            Some(time) => println!("    リンクした時刻（UNIX 秒）: {time}"),
            None => println!("    リンクした時刻: 読めません（PE 形式ではない）"),
        }
        if !profile.is_empty() {
            let found = count(&bytes, profile.as_bytes());
            println!("    ユーザーフォルダの場所が入っている数: {found} か所");
        }
        files.push(bytes);
    }
```

```
    if let [first, second] = files.as_slice() {
        let same_length = first.len() == second.len();
        let differ = first.iter().zip(second).filter(|(a, b)| a != b).count();
        println!("--- 2つを比べる");
        println!("    大きさが同じ: {same_length}");
        println!("    違うバイトの数: {differ}");
    }
    Ok(())
}
```

同じソースから、release ビルドを2回作って比べます。2回目は、RustNote の部分だけを消してから作り直します（依存クレートは作り直さないので、1回目より速く終わります）。

```
# 1回目の実行ファイルを、builds フォルダに取っておく
New-Item -ItemType Directory builds
Copy-Item target\release\rustnote.exe builds\build1.exe
# RustNote の部分だけを消して、作り直す
cargo clean --release -p rustnote
cargo build --release --locked
Copy-Item target\release\rustnote.exe builds\build2.exe
# 2つを比べる
cargo run -q --example u01_exe_report -- builds/build1.exe builds/build2.exe
```

▼ 実行結果

```
[builds/build1.exe]
    大きさ: 11732480 バイト
    リンクした時刻（UNIX 秒）: 1793398680
    ユーザーフォルダの場所が入っている数: 1811 か所
[builds/build2.exe]
    大きさ: 11732480 バイト
    リンクした時刻（UNIX 秒）: 1793398806
    ユーザーフォルダの場所が入っている数: 1811 か所
--- 2つを比べる
    大きさが同じ: true
    違うバイトの数: 24
```

▼ コードを1つずつ分解して解説

解説1: 同じソースなのに、24 バイト違った

- 2つの実行ファイルは、大きさは同じですが、中身が 24 バイト違いました。SHA-256 の指紋（`Get-FileHash` で表示できる）も、まったく別の値になります。

- 違いは、**リンクした時刻**でした。1793398680 と 1793398806 の差は 126 秒で、この PC で2回のビルドの間にたった時間です。Windows の実行ファイル（PE 形式）の見出しには、作った時刻が入ります。違う 24 バイトの場所を1つずつ調べると、見出しの時刻（2 バイト）、デバッグ情報の目次に入っている同じ時刻（2 バイトずつ3か所）、デバッグ情報のファイル（`rustnote.pdb`）と組み合わせるための識別番号（16 バイト）でした。プログラムの命令の部分は、1 バイトも違いませんでした。
- つまり、**コードは同じでも、作るたびに指紋が変わる**ということです。これでは、「公開した実行ファイルが、本当にこのソースから作った物か」を、もう一度作って指紋を比べる方法で確かめられません。

解説2: /Brepro を付けると、同じ中身になる

- Windows のリンカー（`link.exe`）には、時刻の代わりに中身から計算した値を使う `/Brepro` という指定があります。Rust から渡すには、環境変数 `RUSTFLAGS` を使います。`RUSTFLAGS` を変えると、全部のクレートが作り直しになるので、別の `--target-dir` で試します。

```
$env:RUSTFLAGS = "-C link-arg=/Brepro"
cargo build --release --locked --target-dir target\brepro
Copy-Item target\brepro\release\rustnote.exe builds\brepro1.exe
cargo clean --release -p rustnote --target-dir target\brepro
cargo build --release --locked --target-dir target\brepro
Copy-Item target\brepro\release\rustnote.exe builds\brepro2.exe
# 試し終わったら、環境変数を消す
Remove-Item Env:RUSTFLAGS
cargo run -q --example u01_exe_report -- builds/brepro1.exe builds/brepro2.exe
```

```
[builds/brepro1.exe]
    大きさ: 11731968 バイト
    リンクした時刻（UNIX 秒）: 2520511643
    ユーザーフォルダの場所が入っている数: 1811 か所
[builds/brepro2.exe]
    大きさ: 11731968 バイト
    リンクした時刻（UNIX 秒）: 2520511643
    ユーザーフォルダの場所が入っている数: 1811 か所
--- 2つを比べる
    大きさが同じ: true
    違うバイトの数: 0
```

- 違うバイトの数が 0 になりました。「リンクした時刻」は、本当の時刻ではなく、中身から計算した値（2520511643。日付にすると 2049 年になる、ありえない値）になっています。これで、同じソースから作れば同じ指紋になりました。

解説3: 実行ファイルに、自分のフォルダの場所が残っている

- 「ユーザーフォルダの場所が入っている数」は、この PC では 1,811 か所でした。panic のメッセージに使う、ソースファイルの場所（`C:\Users\あなたの名前\.cargo\registry\...` など）が、実行ファイルの中に文字として残っているのです。この実行ファイルを人に渡すと、あなたのユーザー名も一緒に渡ることになります。
- `--remap-path-prefix` を使うと、実行ファイルに入れる場所の文字を、別の文字に置き換えられます。

```
$env:RUSTFLAGS = "-C link-arg=/Brepro --remap-path-prefix=$env:USERPROFILE=~"
cargo build --release --locked --target-dir target\remap
Copy-Item target\remap\release\rustnote.exe builds\remap.exe
Remove-Item Env:RUSTFLAGS
cargo run -q --example u01_exe_report -- builds/remap.exe
```

```
[builds/remap.exe]
    大きさ: 11708416 バイト
    リンクした時刻（UNIX 秒）: 2255404590
    ユーザーフォルダの場所が入っている数: 1 か所
```

- 1,811 か所が 1 か所に減りました。残った1か所は、RustNote のフォルダの場所です。Topcoat の `asset!` マクロは、`assets/style.css` のような場所を、あとで topcoat CLI が見つけられるように、**クレートのフォルダの場所**（`CARGO_MANIFEST_DIR`）を実行ファイルに書き込みます（topcoat-asset 0.11.0 の `asset!` のソースで確かめました）。これは、アセットの束を作るために必要な情報です。
- この単元の実験で分かったのは、**作る場所（フォルダの場所・ユーザー名・時刻）が、実行ファイルの中身に入り込む**ということです。そこで、本番用の実行ファイルは、自分の PC ではなく、**いつも同じ場所・同じ道具で作る**ようにします。RustNote の `Dockerfile` が、`rust:1.99.1-trixie` のイメージの中の `/app` フォルダでビルドしているのは、このためです（UNIT 02（準備中））。



図12-4: 作る場所が実行ファイルに入り込む様子。リンカーとコンパイラーの指定で減らせるが、本番用はいつも同じ場所（Docker の中）で作るのが確実

よくあるエラーと読み方

エラー1: アセットの束がない（no asset bundle at ...）

release ビルドを作った直後に、`topcoat asset bundle --release` をせずに起動した場合です。`--release` を付けずに `topcoat asset bundle` だけを実行した場合も、同じエラーになります（束が `target\debug\assets` にできて、release の実行ファイルの隣には無いからです）。

```
topcoat asset bundle
.\target\release\rustnote.exe
```

```
bundled assets into target\debug\assets
```

```
Error: アセットの束（assets/）がありません。先に `topcoat asset bundle` を実行するか、`topcoat dev` で起動してください

Caused by:
    no asset bundle at target\release\assets
（終了コード: 1）
（このあとのフォルダの中: ）
```

- **何が起きたか:** `Error:` の行は、RustNote の `main.rs` が `anyhow` の `context` で付けた説明です。`Caused by:` の下の `no asset bundle at target\release\assets` が、Topcoat の `AssetBundle::load()` が返した、もとの原因です。実行ファイルの隣（`target\release\assets`）に束がありませんでした。
- **DB には触っていない:** RustNote の `main.rs` は、アセットを読み込んでから DB を開きます。だから、このエラーのときは、`rustnote.db` も作られていません。準備に失敗するものを、先に確かめているのです（第9巻 UNIT 06）。
- **直し方:** `topcoat asset bundle --release` を実行してから、もう一度起動します。
- **RustNote の改善点:** エラーの文は「先に `topcoat asset bundle` を実行するか」と案内していますが、release の実行ファイルのときは `--release` も必要です。「release のときは `--release` を付ける」まで書いておくと、迷う人が減ります（RustNote のコードは変えていません）。

エラー2: ポートがもう使われている（os error 10048）

RustNote が動いているのに、別の窓から、もう1つ起動した場合です。

```
.\target\release\rustnote.exe
```

```
RustNote を起動しました: http://127.0.0.1:3000
Error: サーバーを起動できませんでした（HOST / PORT を確認してください）

Caused by:
    通常、各ソケット アドレスに対してプロトコル、ネットワーク アドレス、またはポートのどれか 1 つのみを使用できます。 (os error 10048)
（終了コード: 1）
```

- **何が起きたか:** `通常、各ソケット アドレスに対して…1 つのみを使用できます。(os error 10048)` は、Windows の「そのアドレスとポート（127.0.0.1:3000）は、もうほかのプログラムが使っています」というエラーです。Linux では `Address already in use (os error 98)` のような文になります。
- **直し方:** 先に動いている RustNote を Ctrl+C で止めるか、`$env:PORT = "3001"` のように、別のポートを指定してから起動します（`topcoat::start` は `HOST` と `PORT` の環境変数を読みます）。
- **気をつけたい点:** 1行目に `RustNote を起動しました: http://127.0.0.1:3000` と出ています。ところが、このプログラムは起動できずに止まりました。RustNote の `main.rs` は、ポートを押さえる（`topcoat::start`）**前に**「起動しました」を表示しているからです。ログを見る人が「起動した」と勘違いしないよう、ポートを押さえたあとに表示するのが本来の形です。これも、RustNote に残っている改善点です（教材のコードは変えていません。直し方は、第13巻の総合演習2で確かめています）。

エラー3（エラーが出ない違い）：整数のあふれが、release では折り返す

`u8`（0〜255 の整数）の 255 に 1 を足す、小さなプログラムです（`examples/u01_overflow.rs`）。

```
//! 整数のあふれ（オーバーフロー）が、debug と release でどう違うかを確かめる。
//!   cargo run -q --example u01_overflow
//!   cargo run -q --release --example u01_overflow

use std::hint::black_box;

fn main() {
    // black_box は、コンパイラに値を先に計算させないための箱（実行してから値が決まる、という扱いにする）
    let count: u8 = black_box(255);
    println!("count = {count}");
    let next = count + 1;
    println!("count + 1 = {next}");
}
```

```
cargo run -q --example u01_overflow
cargo run -q --release --example u01_overflow
```

```
count = 255

thread 'main' (6484) panicked at examples\u01_overflow.rs:11:16:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
count = 255
count + 1 = 0
```

- **何が起きたか:** debug では `attempt to add with overflow`（足し算であふれようとしました）の panic で止まりました。release では、何も言わずに `0` になりました。`u8` の 256 は表せないので、0 に折り返したのです。
- `black_box` を使っているのは、`255 + 1` のように、コンパイルのときに答えが分かる式だと、コンパイラーが先にエラーにしてしまうからです。実際のプログラムでは、値はフォームや DB から来るので、コンパイラーは気づけません。
- **これが怖い理由:** テスト（debug）では panic で気づけたのに、本番（release）では、まちがった数のまま動き続けます。お金や件数の計算で起きると、静かにデータが壊れます。
- **直し方は2通り:** 1つは、あふれる可能性のある計算を、`checked_add`（あふれたら `None`）や、大きい型（`u64`）で計算することです（問2）。もう1つは、`Cargo.toml` に次の設定を書いて、release でもあふれを確かめるようにすることです。少し遅くなるかわりに、debug と同じく panic で止まります。

```
[profile.release]
overflow-checks = true
```

この設定は、`Cargo.toml` を書き換えずに、環境変数 `CARGO_PROFILE_RELEASE_OVERFLOW_CHECKS=true` で試すこともできます。試すと、release でも debug と同じ panic になりました。

```
count = 255

thread 'main' (43852) panicked at examples\u01_overflow.rs:11:16:
attempt to add with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

RustNote ではここで使う

RustNote の `Dockerfile` の「1. builder」の部分は、この単元の手順を、そのまま並べた形です（抜き出しています）。

```
# builder と実行用は、同じ Debian 13（trixie）にそろえる（実行ファイルが使う glibc の版を合わせるため）
FROM rust:1.99.1-trixie AS builder
WORKDIR /app

# アセットの束を作る topcoat CLI（アプリの Topcoat と同じ 0.11.0）
RUN cargo install topcoat-cli --version 0.11.0 --locked

# ソースをコピーしてビルドする（Cargo.lock の版をそのまま使う）
COPY . .
RUN cargo build --release --locked

# 実行ファイルの隣（target/release/assets/）に CSS の束を作る
RUN topcoat asset bundle --release
```

- Rust の版（1.99.1）と、作るフォルダ（`/app`）が、いつも同じです。例1-3で見た「作る場所が中身に入り込む」問題が、毎回同じ形になります。
- `cargo build --release --locked` → `topcoat asset bundle --release` の順番も、例1-1と同じです。

整数のあふれについても、RustNote は気をつけて書いてあります。`src/domain.rs` の `NoteQuery` です（抜き出しています）。

```
            page: page.unwrap_or(1).max(1),
```

```
    pub fn offset(&self) -> u64 {
        u64::from(self.page - 1) * u64::from(PER_PAGE)
    }
```

- `page.unwrap_or(1).max(1)` で、ページ番号を必ず 1 以上にしています。だから `self.page - 1` が 0 より小さくなる（`u32` で引き算があふれる）ことはありません。`?page=0` が来ても、1 ページ目として扱います。
- 何件飛ばすか（`offset`）は、`u32` のままではなく `u64` に広げてから掛けています。`u32` の最大値どうしを掛けても、`u64` ならあふれません。なぜこう書くのかは、問2で確かめます。

練習問題

問1: release ビルドの `rustnote.exe` だけを、別のフォルダ（たとえば `dist`）にコピーして起動すると、どうなるでしょうか。予想してから確かめてください。そのあと、そのフォルダで RustNote を起動できるようにするには、何をコピーすればよいでしょうか。起動できたら、DB ファイルはどこにできたかも確かめてください。

問2: もし `NoteQuery::offset` を `u64` に広げずに、`u32` のまま `(self.page - 1) * PER_PAGE` と書いていたら、URL に `?page=214748366` を付けて開いたとき、debug と release でそれぞれどうなるでしょうか。小さなプログラムで確かめてください。

問3: 公開した実行ファイルについて、「どのコミット・どの Rust・どの `Cargo.lock` から作ったか」を後から確かめられるように、PowerShell で記録のファイル（`release-record.txt`）を作ってください。練習用のフォルダを Git で管理している（`git init` してコミットしてある）ものとします。

解答と解説

問1の解答:

予想: 起動しない。実行ファイルは「自分の隣の `assets\`」を読むのに、`dist` には `assets\` が無いからです。

```
New-Item -ItemType Directory dist
Copy-Item target\release\rustnote.exe dist\
cd dist
.\rustnote.exe
```

```
Error: アセットの束（assets/）がありません。先に `topcoat asset bundle` を実行するか、`topcoat dev` で起動してください

Caused by:
    no asset bundle at ...\dist\assets
（終了コード: 1）
```

- 今いるフォルダではなく、**実行ファイルのある場所**（`dist\assets`）を探しました。`target\release\assets` があっても、使われません。
- `target\release\assets` を、フォルダごと `dist` にコピーすると起動できます。`Copy-Item -Recurse ..\target\release\assets .\assets` のあと、もう一度起動して `/health` を開くと、`{"status":"ok"}` が返りました。

```
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://127.0.0.1:3000
（/health: {"status":"ok"}）
（dist の中: assets rustnote.db rustnote.exe ）
```

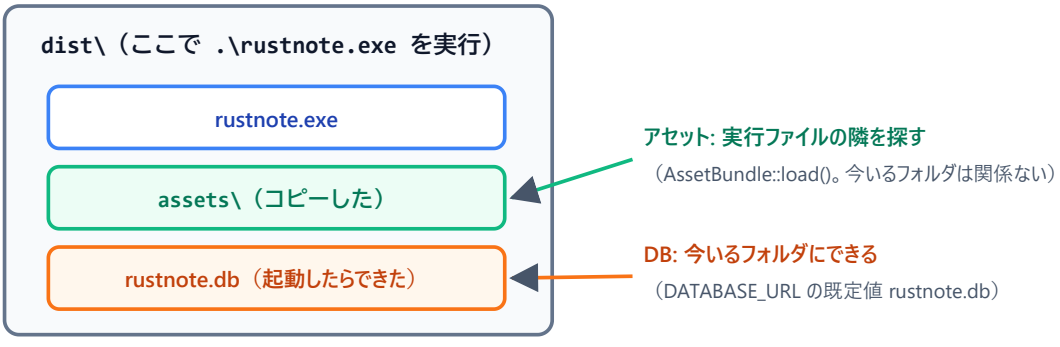


図12-5: 探す場所の決まりの違い。アセットは実行ファイルの隣を、DB ファイルは今いるフォルダを使う

- 起動したあとの `dist` には、`rustnote.exe`・`assets`・`rustnote.db` の3つがありました。DB ファイルは、今いるフォルダ（`dist`）にできます。アセットは「実行ファイルの隣」、DB は「今いるフォルダ」と、探す場所の決まりが違うことに注意してください。本番では、DB の場所を `DATABASE_URL` で決めておきます（RustNote の `Dockerfile` は `/data/rustnote.db`）。

問2の解答:

`examples/u01_q2.rs` です。RustNote と同じ `u64` の計算と、`checked_mul`、`u32` のままのふつうの `*` を並べます。

```
//! 問2の解答: ページ番号から「何件飛ばすか」（offset）を計算するとき、
//! u32 のままだとあふれる。RustNote の NoteQuery::offset と比べる。
//!   cargo run -q --example u01_q2
//!   cargo run -q --release --example u01_q2

use std::hint::black_box;

const PER_PAGE: u32 = 20;

fn main() {
    // URL の ?page=214748366 から来たページ番号
    let page: u32 = black_box(214_748_366);

    // RustNote と同じ: u64 に広げてから掛ける（u32 の最大値どうしを掛けてもあふれない）
    let wide = u64::from(page - 1) * u64::from(PER_PAGE);
    println!("u64 に広げて計算: {wide}");

    // checked_mul は、あふれたら None を返す
    println!(
        "u32 のまま checked_mul: {:?}",
        (page - 1).checked_mul(PER_PAGE)
    );

    // u32 のまま、ふつうの * で掛ける
    let narrow = (page - 1) * PER_PAGE;
    println!("u32 のまま *: {narrow}");
}
```

```
cargo run -q --example u01_q2
cargo run -q --release --example u01_q2
```

```
u64 に広げて計算: 4294967300
u32 のまま checked_mul: None

thread 'main' (219748) panicked at examples\u01_q2.rs:25:18:
attempt to multiply with overflow
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

```
u64 に広げて計算: 4294967300
u32 のまま checked_mul: None
u32 のまま *: 4
```

- 正しい答えは 4,294,967,300 件飛ばすこと（`u64` の結果）です。これは `u32` の最大値（4,294,967,295）を少しだけ超えます。
- debug では、掛け算があふれて panic しました。RustNote のページの処理の中なら、500 の返事になります（第9巻 UNIT 04 の寄り道）。
- release では、`4` になりました。4,294,967,300 から 4,294,967,296 を引いた余りです。つまり、**とても大きなページ番号を付けただけで、5 件目からのノートが表示されてしまう**のです。エラーにもならないので、気づくのはとても難しくなります。
- `checked_mul` は、あふれるときに `None` を返すので、「あふれた」ことをプログラムで扱えます。RustNote のように、はじめから大きな型で計算するのも、確実な方法です。

問3の解答:

```powershell
$exe = "target\release\rustnote.exe"
@(
    "commit:     $(git rev-parse --short HEAD)"
    "rustc:      $(rustc --version)"
    "Cargo.lock: $((Get-FileHash Cargo.lock).Hash)"
    "rustnote:   $((Get-FileHash $exe).Hash)"
    "built at:   $((Get-Item $exe).LastWriteTimeUtc.ToString('u'))"
) | Set-Content -Encoding utf8 release-record.txt
Get-Content release-record.txt
```

```
commit:     6b40498
rustc:      rustc 1.99.1 (58cf8f9c3 2026-10-13)
Cargo.lock: FC84B1A4DFA47B85BD2C49F52BF522EBDC3E0B948846FA2862050904E9B000E0
rustnote:   36E406CFEF0D4AC42BB6D25F2007C6A05860BDBF875CE0E2C8C2CEB2F0CD9C1E
built at:   2026-10-30 22:20:06Z
```

- `git rev-parse --short HEAD` は、今のコミットの短い名前です。`rustc --version` は、このフォルダで使われる Rust の版（`rust-toolchain.toml` の 1.99.1）です。`Get-FileHash` は、ファイルの SHA-256 の指紋を計算します。
- `built at` は、実行ファイルを最後に書き換えた時刻（UTC）です。この例では、例1-3で調べた2回目のビルドの「リンクした時刻」（1793398806 秒）と同じ、2026-10-30 22:20:06（UTC）になりました。
- この記録があれば、本番で問題が起きたとき、「このコミットを、この Rust と、この `Cargo.lock` で作った物」とたどれます。`/Brepro` などで再現できるように作ってあれば、同じコミットからもう一度作って、`rustnote` の指紋が一致するかで確かめることもできます。

- 記録のファイルは、Git に入れるより、実行ファイルと一緒に保管するほうが向いています（コミットの名前を、そのコミットの中に書くことはできないからです）。

まとめ

- 公開するのは release ビルド。この PC では、debug より大きさが約半分、パスワードのハッシュ化が約 20 倍速かった。ただし release は整数のあふれを確かめないので、大きい型や `checked_*` で計算するか、`overflow-checks = true` を設定する。
- 本番用の実行ファイルは、`cargo build --release --locked` → `topcoat asset bundle --release` の順で作り、実行ファイルと `assets\` を必ず一緒に置く。実行ファイルは「自分の隣の `assets\` 」を、DB は「今いるフォルダ」（または `DATABASE_URL` ）を使う。
- 作る時刻と場所は、実行ファイルの中身に入り込む。同じソースから同じ物を作れるよう、作る場所と道具を固定し（Docker）、コミット・Rust の版・`Cargo.lock` ・実行ファイルの指紋を記録しておく。

UNIT 02: multi-stage の Dockerfile

この単元のゴール

- RustNote の `Dockerfile` の2つの段（作る段と動かす段）が、それぞれ何を入れ、何を入れないかを、本物のビルドと `docker history` で説明できる
- 1段だけの Dockerfile と比べて、イメージの大きさ・入っている物・動かす利用者が、どう違うかを言える
- Docker のキャッシュが段ごとに効くことと、ソースを1行変えたときに何がやり直しになるかを、ビルドの記録で読める。キャッシュのマウントで、やり直しを短くできる
- 実行用の段の決まり（同じ Debian にそろえる・`COPY --from` の場所・`/data` の持ち主）を守らないと何が起きるかを、エラーで確かめられる

たとえ話でつかむ

パン屋さんの、**工房**と**お店の棚**を考えます。

- 工房（builder の段）には、オーブン・こね台・粉の袋が、全部そろっています。パンを焼くには、どれも要ります。
- お店の棚（実行用の段）に並べるのは、**焼き上がったパンと、値札だけ**です。オーブンや粉の袋まで棚に置いたら、場所を取るうえに、お客さんが勝手に触ってしまうかもしれません。
- 店員さん（利用者 `app` ）は、棚の商品（ `/app` ）を作り替えることはできず、**在庫の置き場**（ `/data` ）だけを使います。
- 工房とお店の**電気の方式**（Linux の種類と glibc）は、そろえておきます。少し古い方式のお店でも動くことはありますが、約束はありません。方式がまるで違うお店では、まったく動きません（エラー1）。
- 粉の袋の中身（依存クレート）が変わらないなら、**仕込みの途中までを取っておく**と、毎回はじめからこねなくて済みます（キャッシュ）。

パン屋さん	RustNote の `Dockerfile`
工房	`FROM rust:1.99.1-trixie AS builder`（Rust と topcoat CLI が入った大きな段）
焼き上がったパンと値札	実行ファイル `rustnote` と、CSS の束 `assets/`
お店の棚	`FROM debian:trixie-slim`（小さな Debian の段）
工房から棚へ運ぶ	`COPY --from=builder …`（要る物だけを運ぶ）
店員さん	利用者 `app`（uid 10001。root ではない）
在庫の置き場	`/data`（DB のファイルを置く、`app` が書ける唯一の場所）
電気の方式をそろえる	builder と実行用を、同じ Debian 13（trixie）にする

> **用語:** multi-stage ビルド（マルチステージ・ビルド：1つの Dockerfile に `FROM` を2つ以上書き、前の段で作った物の中から、要る物だけを次の段へ `COPY --from` で運ぶ作り方）

> **用語:** レイヤー（Docker のイメージを作る1つ1つの段の結果。`RUN`・`COPY` ごとに1つでき、同じ手順・同じ材料なら、前のビルドの結果（キャッシュ）を使い回す）

> **用語:** glibc（ジー・リブ・シー：Linux の C の基本のライブラリ。Rust の実行ファイルも、ふつうはこれを使う。新しい glibc で作った実行ファイルは、古い glibc の上では動かないことがある）

図で理解する

教材の `Dockerfile` の2つの段と、それぞれに入る物です（例2-1・例2-2の結果）。

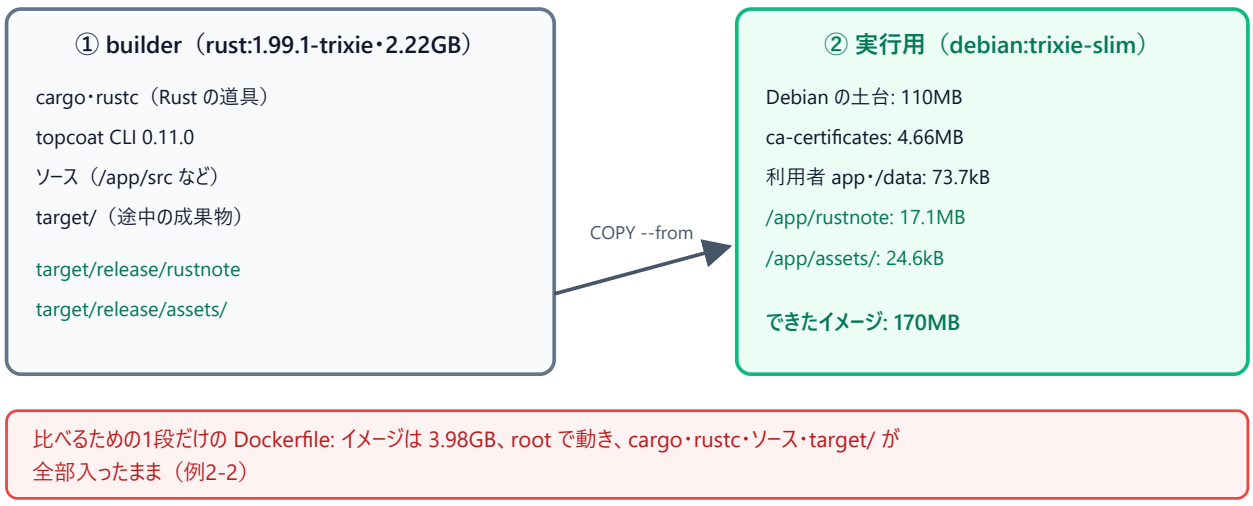


図12-6: 2つの段。大きな builder の段で作り、実行用の段には、実行ファイルと assets/ の2つだけを運ぶ。数は docker history と docker image ls の結果

ソースを1行変えたとき、どの段がやり直しになるかです（例2-3の結果）。

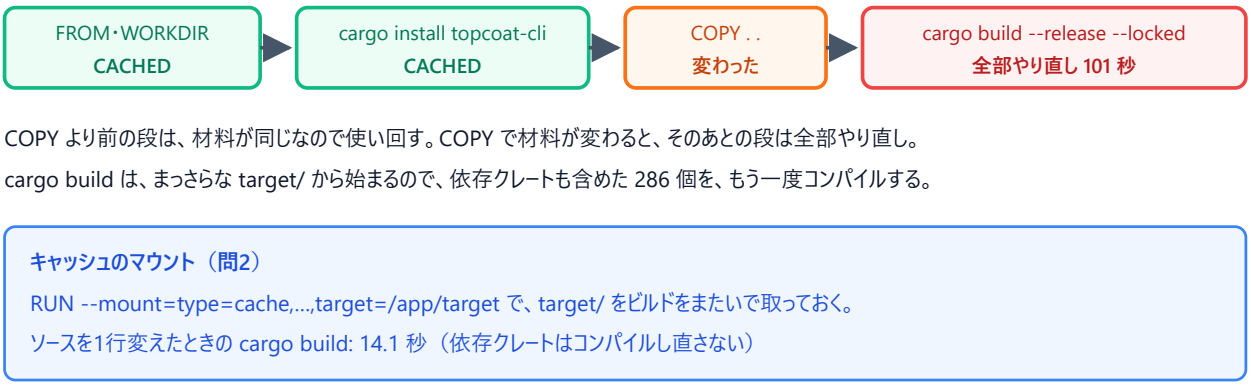


図12-7: Docker のキャッシュは段ごと。材料（COPY）が変わった段から下は全部やり直しになる。target/ をキャッシュのマウントで残すと、Rust のビルドは変わった所だけで済む

まず動かす最小コード

例2-1: 教材の Dockerfile でイメージを作る

▼ **このファイルがやること（先に日本語で）**: builder の段で、Rust 1.99.1 の入った Debian 13 のイメージに topcoat CLI を入れ、ソースをコピーして release ビルドとアセットの束を作ります。実行用の段では、小さな Debian に証明書と利用者 `app` と `/data` を用意し、builder の段から実行ファイルと `assets/` だけをコピーして、`0.0.0.0:8080` で動かします。

```
# RustNote のコンテナイメージ（Fly.io で動かす）
#
# 2 段階（multi-stage）ビルド:
#   1. builder … Rust と topcoat CLI でアプリをビルドし、CSS の束（assets/）を作る
#   2. 実行用  … 小さな Debian に、実行ファイルと assets/ だけをコピーする
#
# ビルド: docker build -t rustnote .
# 実行:   docker run --rm -p 8080:8080 rustnote

# ---- 1. builder ----
# builder と実行用は、同じ Debian 13（trixie）にそろえる（実行ファイルが使う glibc の版を合わせるため）
FROM rust:1.99.1-trixie AS builder
WORKDIR /app

# アセットの束を作る topcoat CLI（アプリの Topcoat と同じ 0.11.0）
RUN cargo install topcoat-cli --version 0.11.0 --locked

# ソースをコピーしてビルドする（Cargo.lock の版をそのまま使う）
COPY . .
RUN cargo build --release --locked

# 実行ファイルの隣（target/release/assets/）に CSS の束を作る
RUN topcoat asset bundle --release

# ---- 2. 実行用 ----
FROM debian:trixie-slim

# Turso へ HTTPS でつなぐための証明書
RUN apt-get update \
    && apt-get install -y --no-install-recommends ca-certificates \
    && rm -rf /var/lib/apt/lists/*

# root ではない利用者で動かす。/data はローカルの DB ファイル置き場
RUN useradd --create-home --uid 10001 app \
    && mkdir -p /data \
    && chown app:app /data

WORKDIR /app
# AssetBundle::load() は「実行ファイルの隣の assets/」を読むので、同じ並びでコピーする
COPY --from=builder /app/target/release/rustnote /app/rustnote
COPY --from=builder /app/target/release/assets /app/assets

USER app

# 0.0.0.0:8080 で待ち受ける（fly.toml の internal_port と合わせる）
ENV HOST=0.0.0.0 \
    PORT=8080 \
    DATABASE_URL=/data/rustnote.db
```

```
EXPOSE 8080

CMD ["/app/rustnote"]
```

教材のフォルダの中身（`.dockerignore` も含む）をコピーしたフォルダで、ビルドしました。

```
docker build --progress=plain -t rustnote:u02 .
```

```
Docker 29.3.1 (linux/arm64)
```

```

```

```
#1 [internal] load build definition from Dockerfile
#1 DONE 0.0s
#2 [internal] load metadata for docker.io/library/debian:trixie-slim
#2 DONE 0.2s
#3 [internal] load metadata for docker.io/library/rust:1.99.1-trixie
#3 DONE 0.2s
#4 [internal] load .dockerignore
#4 transferring context: 143B done
#4 DONE 0.0s
#5 [internal] load build context
#5 DONE 0.0s
#6 [stage-1 1/6] FROM docker.io/library/debian:trixie-
slim@sha256:f6b08b09a05725f9431502d3b5714f5c4576e82c1a84fa1debe0c24a7e0c3ec4
#6 DONE 0.1s
#7 [builder 1/6] FROM docker.io/library/rust:1.99.1-
trixie@sha256:09027f873d45b8ecd1dc86f1fa13b7a62b875c961f017a0e331267b350430137
#7 DONE 0.1s
#5 transferring context: 244.18kB 0.0s done
#5 DONE 0.1s
#8 [builder 2/6] WORKDIR /app
#8 CACHED
#9 [builder 3/6] RUN cargo install topcoat-cli --version 0.11.0 --locked
#9 CACHED
#10 [builder 4/6] COPY . .
#10 DONE 0.0s
#11 [builder 5/6] RUN cargo build --release --locked
#11 DONE 86.8s
#12 [builder 6/6] RUN topcoat asset bundle --release
#12 DONE 1.1s
#13 [stage-1 5/6] COPY --from=builder /app/target/release/rustnote /app/rustnote
#13 CACHED
#14 [stage-1 2/6] RUN apt-get update     && apt-get install -y --no-install-recommends
ca-certificates     && rm -rf /var/lib/apt/lists/*
#14 CACHED
#15 [stage-1 3/6] RUN useradd --create-home --uid 10001 app     && mkdir -p /data
&& chown app:app /data
#15 CACHED
#16 [stage-1 4/6] WORKDIR /app
#16 CACHED
#17 [stage-1 6/6] COPY --from=builder /app/target/release/assets /app/assets
#17 CACHED
#18 DONE 0.2s
```

▼ コードを1つずつ分解して解説

解説1: 2つの段は、並んで進む

- 記録の `[builder 3/6]` は builder の段の3つ目の手順、`[stage-1 2/6]` は実行用の段（名前を付けていないので `stage-1` ）の2つ目の手順です。2つの段は、互いに待たなくてよい所は、同時に進みます。実行用の段の `COPY --from=builder` だけは、builder の段が終わるのを待ちます。
- `CACHED` は、前のビルド（第13巻 UNIT 10）の結果を、そのまま使ったという意味です。この PC では、`cargo install topcoat-cli` も、実行用の段の準備も、前に作ってあったので、やり直したのは `COPY . .` からあとだけでした。
- `cargo build --release --locked` は 86.8 秒でした。そのあとの `COPY --from=builder … /app/rustnote` まで `CACHED` だったのは、できた実行ファイルが、前のビルドと**1バイトも違わなかった**からです（UNIT 01 で見た再現性。Docker の中では、作る場所と道具がいつも同じ）。

解説2: 実行用の段の中身を見る

`docker history` で、できたイメージのレイヤーと大きさを、上（新しい）から並べました。

```
docker history rustnote:u02 --format "{{.Size}}`t{{.CreatedBy}}"
```

```
0B  CMD ["/app/rustnote"]
0B  EXPOSE [8080/tcp]
0B  ENV HOST=0.0.0.0 PORT=8080 DATABASE_URL=/dat…
0B  USER app
24.6kB  COPY /app/target/release/assets /app/assets …
17.1MB  COPY /app/target/release/rustnote /app/rustn…
8.19kB  WORKDIR /app
73.7kB  RUN /bin/sh -c useradd --create-home --uid 1…
4.66MB  RUN /bin/sh -c apt-get update     && apt-get…
110MB   # debian.sh --arch 'arm64' out/ 'trixie' '@1…
```

```
rustnote:u02 の大きさ: 170MB
rust:1.99.1-trixie の大きさ: 2.22GB
debian:trixie-slim の大きさ: 142MB
```

- いちばん大きいのは、Debian の土台（110MB）です。RustNote の実行ファイルは 17.1MB、CSS の束は 24.6kB、証明書は 4.66MB でした。builder の段の 2GB 以上は、どこにも入っていません。
- `docker image ls` の 170MB と、`docker history` の合計が少し違うのは、数え方（圧縮・メタデータ）の違いです。

イメージの中で、だれとして動き、どこに書けるかを確かめました。

```
docker run --rm rustnote:u02 sh -c 'id; ls -la /app /data; touch /app/x; touch /data/x
&& echo "/data に書けた"'
```

```
uid=10001(app) gid=10001(app) groups=10001(app)
/app:
total 16752
drwxr-xr-x 1 root root     4096 Oct 30 21:40 .
drwxr-xr-x 1 root root     4096 Oct 30 22:30 ..
drwxr-xr-x 2 root root     4096 Oct 30 21:40 assets
-rwxr-xr-x 1 root root 17140968 Oct 30 21:40 rustnote

/data:
total 8
drwxr-xr-x 2 app  app  4096 Oct 30 21:37 .
drwxr-xr-x 1 root root 4096 Oct 30 22:30 ..
touch: cannot touch '/app/x': Permission denied
/data に書けた
```

- 利用者は `app`（uid 10001）です。`/app` の中（実行ファイルと assets）は root の物なので、`app` は書き換えられません（`Permission denied`）。書けるのは、`chown app:app` した `/data` だけです。
- もし RustNote に、ファイルを書き換えられてしまうような弱点があっても、実行ファイルそのものは書き換えられません。権限を必要な分だけにしておく考え方です（最小権限）。

もう一歩: 実務に近い形

例2-2: 1段だけの Dockerfile と比べる

比べるために、作る道具ごとそのまま動かす、1段だけの Dockerfile を書いて、ビルドしました。

```
# 比べるための、1段だけの Dockerfile（作る道具ごと、そのまま動かす）
FROM rust:1.99.1-trixie
WORKDIR /app
RUN cargo install topcoat-cli --version 0.11.0 --locked
COPY . .
RUN cargo build --release --locked && topcoat asset bundle --release
ENV HOST=0.0.0.0 PORT=8080 DATABASE_URL=/app/rustnote.db
CMD ["/app/target/release/rustnote"]
```

```
```

```
rustnote:single の大きさ: 3.98GB
uid=0(root) gid=0(root) groups=0(root)
/usr/local/cargo/bin/cargo
/usr/local/cargo/bin/rustc
Cargo.lock
Cargo.toml
Dockerfile
assets
migrations
rust-toolchain.toml
rustfmt.toml
src
target
tests
```

- イメージは **3.98GB** で、2段のイメージ（170MB）の 20 倍以上でした。Rust の道具（2.22GB）に、`target/` の途中の成果物まで、全部入っているからです。Fly.io に送るたび、起動するたびに、この大きさを運ぶことになります。
- **root** で動き、`cargo` と `rustc` が入ったまま、`/app` にはソース・`tests`・`target` が残っています。万一、中に入り込まれたら、その場でプログラムを作って動かせてしまいます。

例2-3: ソースを1行変えて、ビルドし直す

`src/routes/health.rs` の最後に、コメントを1行足して、教材の Dockerfile でビルドし直しました。

```

```

```
#8 [builder 2/6] WORKDIR /app
#8 CACHED
#9 [builder 3/6] RUN cargo install topcoat-cli --version 0.11.0 --locked
#9 CACHED
#10 [builder 4/6] COPY . .
#10 DONE 0.3s
#11 [builder 5/6] RUN cargo build --release --locked
#11 DONE 101.1s
#12 [builder 6/6] RUN topcoat asset bundle --release
#12 DONE 2.6s
```

- `COPY . .` より前（`WORKDIR`・`cargo install topcoat-cli`）は `CACHED` でした。`COPY . .` で材料が変わったので、そのあとの `cargo build` は、依存クレートを含めて全部やり直しになり、101.1 秒かかりました（`Compiling` の行が 286 行）。
- `cargo build` は、レイヤーの中で、まっさらな `target/` から始まるからです。手元の PC の `cargo build` なら、変わった RustNote の部分だけをコンパイルし直すので、数秒で終わります。

- ビルドの記録には、`info: downloading 2 components` という行もありました。`rust-toolchain.toml` の `components = ["rustfmt", "clippy"]` のため、イメージに入っていない rustfmt と clippy を、ビルドのたびに取りに行っているのです。実行ファイルを作るだけなら要らない物です（改善点）。

よくあるエラーと読み方

エラー1: 実行用の段を alpine にした（exec … no such file or directory）

小さなイメージとしてよく使われる alpine を、実行用の段に使ってみました（alpine には `useradd` が無いので、利用者を作る行も alpine の書き方に直しました）。

```
--- a/Dockerfile
+++ b/Dockerfile
@@ -23,15 +23,13 @@
 RUN topcoat asset bundle --release

 # ---- 2. 実行用 ----
-FROM debian:trixie-slim
+FROM alpine:3.22

 # Turso へ HTTPS でつなぐための証明書
-RUN apt-get update \
-    && apt-get install -y --no-install-recommends ca-certificates \
-    && rm -rf /var/lib/apt/lists/*
+RUN apk add --no-cache ca-certificates

 # root ではない利用者で動かす。/data はローカルの DB ファイル置き場
-RUN useradd --create-home --uid 10001 app \
+RUN adduser -D -u 10001 app \
     && mkdir -p /data \
     && chown app:app /data
```

```
```

```
exec /app/rustnote: no such file or directory
（終了コード: 255）
```

- **何が起きたか:** ビルドは通りましたが、起動すると `exec /app/rustnote: no such file or directory`（そのファイルがありません）で止まりました。ところが、ファイルはあります。

```
-rwxr-xr-x    1 root     root      17140968 Oct 30 23:02 /app/rustnote
ls: /lib/ld-linux-aarch64.so.1: No such file or directory
/lib/ld-musl-aarch64.so.1
```

- **読み方:** 実行ファイルは `/app/rustnote` にあります。無いのは、実行ファイルを読み込むための道具（glibc の `ld-linux-aarch64.so.1`）です。alpine は glibc ではなく musl という別の C のライブラリを使うので、glibc 向けに作った RustNote の実行ファイルは動きません。「ファイルが無い」という文は、実行ファイルそのものではなく、それを読み込む道具が無いという意味でした。
- **直し方:** 実行用の段は、builder の段と同じ Debian（`debian:trixie-slim`）にします。どうしても alpine にしたいなら、builder の段も musl 向け（この PC の Docker なら `aarch64-unknown-linux-musl` のターゲット）で作ります。

比べるために、実行用の段だけを1つ古い Debian 12（`debian:bookworm-slim`。glibc 2.36）にしたイメージも作りました。

```
実行用の段: ldd (Debian GLIBC 2.36-9+deb12u14) 2.36
/health: {"status":"ok"}
--- ログ
マイグレーションを適用しました: 0001_users
マイグレーションを適用しました: 0002_notes
マイグレーションを適用しました: 0003_sessions
RustNote を起動しました: http://0.0.0.0:8080
```

- こちらは、glibc 2.41 の trixie で作った実行ファイルが、glibc 2.36 の上でも、エラー無く動きました。RustNote が、新しい glibc にしか無い機能を使っていなかったからです。ただし、これはたまたまで、Rust や依存クレートの版が上がると、`GLIBC_2.38 not found` のようなエラーで止まることがあります。だから、builder と実行用は、同じ版にそろえておきます（`Dockerfile` の先頭のコメントのとおり）。

エラー2: `COPY --from` の場所を間違えた

実行用の段の `COPY --from=builder /app/target/release/rustnote` を、`/app/target/rustnote`（`release` が抜けた）にしてビルドしました。

```
#16 ERROR: failed to calculate checksum of ref
dh3liq5heezeb2pggtmsep689::pegkidym1va49ix7l4gdsfjk1: "/app/target/rustnote": not found
ERROR: failed to build: failed to solve: failed to compute cache key: failed to
calculate checksum of ref dh3liq5heezeb2pggtmsep689::pegkidym1va49ix7l4gdsfjk1:
"/app/target/rustnote": not found
```

- **何が起きたか:** ビルドの最後の段で、「`/app/target/rustnote` が見つかりません（not found）」と止まりました。builder の段の中に、その場所のファイルが無いからです。
- **読み方:** `failed to calculate checksum of ref …` は、Docker がコピーする物の指紋を計算しようとして、物が無かった、という意味です。止まるのはビルドのときなので、壊れたイメージが公開されることはありません。CI の `docker build` で、毎回確かめられます。
- **直し方:** builder の段で、物がどこにできるかを確かめます（release ビルドは `target/release/`）。

エラー3: `/data` を作らずに、root でない利用者で動かした

実行用の段の `RUN useradd … && mkdir -p /data && chown app:app /data` から、`/data` を作る部分を消してビルドし、起動しました。

```
Error: DB（/data/rustnote.db）を開けませんでした

Caused by:
    Failed to connect to database: `Unable to open connection to local database
/data/rustnote.db: 14`
（終了コード: 1）
```

- **何が起きたか:** ビルドは通りましたが、起動すると「DB（/data/rustnote.db）を開けませんでした」で止まりました。原因の `14` は、SQLite の「ファイルを開けません（SQLITE_CANTOPEN）」の番号です。
- **読み方:** `/data` が無いので、DB のファイルを作る場所がありません。root なら `/` の下に作れるかもしれませんが、利用者 `app` には書く権限がありません。最小権限にしたので、書ける場所を、こちらで用意する必要があるのです。
- **直し方:** `mkdir -p /data && chown app:app /data` で、`app` が書けるフォルダを用意します。Fly.io で volume を使うときも、同じ `/data` に付けます。本番の RustNote は Turso を使うので、`/data` は使いません（UNIT 05・09）。

RustNote ではここで使う

- **CI:** `ci.yml` の `docker` のジョブは、テストが通ったあとに `docker build -t rustnote Rust/rustnote` をします。Dockerfile が壊れていないか（エラー2のような書き間違い）を、毎回確かめられます（第13巻 UNIT 10）。
- **Fly.io:** `fly deploy` は、`fly.toml` の `[build] dockerfile = "Dockerfile"` で、この Dockerfile からイメージを作って公開します（UNIT 04・08）。
- **改善点（この単元で見つかったこと）:**
- ソースを1行変えるだけで、依存クレートまで全部コンパイルし直します（例2-3）。キャッシュのマウントで短くできます（問2）。
- `rust-toolchain.toml` の `components` のため、ビルドのたびに rustfmt と clippy をダウンロードしています。

練習問題

問1: 教材の Dockerfile で、`RUN cargo install topcoat-cli …` を `COPY . .` より**前**に書いているのは、なぜでしょうか。例2-3の記録から考えてください。

問2: ソースを1行変えたときのビルドを短くするために、`cargo build` の `target/` を、ビルドをまたいで取っておく（キャッシュのマウント）ように、Dockerfile を直してください。直す前と後で、時間を比べます。

問3: 例2-1で、`/app` に書けず、`/data` には書けました。もし実行用の段で `USER app` を書き忘れると、どうなるでしょうか。

解答と解説

問1の解答:

- Docker は、段を上から順に見て、「手順も材料も前と同じ」ならキャッシュを使います。1つでも変わると、そこから下は全部やり直しです。
- `cargo install topcoat-cli` は、ソースと関係が無く、数分かかる手順です（第13巻 UNIT 10 の最初のビルドでは 98 秒）。`COPY . .` より前に置くと、ソースを変えても `CACHED` のままです（例2-3）。もし `COPY . .` のあとに置いていたら、ソースを1行変えるたびに、topcoat CLI のインストールからやり直しになります。
- 時間のかかる手順で、あまり変わらない物ほど、上に置きます。

問2の解答:

`cargo build` の `RUN` に、ダウンロードしたクレートの置き場（`/usr/local/cargo/registry`）と `target/` を、キャッシュのマウント（`--mount=type=cache`）として付けました。マウントした所はイメージに残らないので、できた実行ファイルと `assets/` は、同じ `RUN` の中で `/out` にコピーし、実行用の段は `/out` から運びます。`id=` は、ほかのプロジェクトのキャッシュと混ざらないための名前です。

```
--- a/Dockerfile
+++ b/Dockerfile
@@ -17,10 +17,15 @@

 # ソースをコピーしてビルドする（Cargo.lock の版をそのまま使う）
 COPY . .
-RUN cargo build --release --locked
-
-# 実行ファイルの隣（target/release/assets/）に CSS の束を作る
-RUN topcoat asset bundle --release
+# ダウンロードしたクレートと target/ を、ビルドをまたいで取っておく（キャッシュのマウント）。
+# マウントした所はイメージに残らないので、できた物は /out にコピーしておく
+RUN --mount=type=cache,id=rustnote-registry,target=/usr/local/cargo/registry \
+    --mount=type=cache,id=rustnote-target,target=/app/target \
+    cargo build --release --locked \
+    && topcoat asset bundle --release \
+    && mkdir -p /out \
+    && cp target/release/rustnote /out/rustnote \
+    && cp -r target/release/assets /out/assets

 # ---- 2. 実行用 ----
 FROM debian:trixie-slim
@@ -37,8 +42,8 @@

 WORKDIR /app
 # AssetBundle::load() は「実行ファイルの隣の assets/」を読むので、同じ並びでコピーする
-COPY --from=builder /app/target/release/rustnote /app/rustnote
-COPY --from=builder /app/target/release/assets /app/assets
+COPY --from=builder /out/rustnote /app/rustnote
+COPY --from=builder /out/assets /app/assets

 USER app
```

```
#11 [builder 5/5] RUN --mount=type=cache,id=rustnote-
registry,target=/usr/local/cargo/registry     --mount=type=cache,id=rustnote-
target,target=/app/target     cargo build --release --locked     && topcoat asset
bundle --release     && mkdir -p /out     && cp target/release/rustnote /out/rustnote
&& cp -r target/release/assets /out/assets
```

ソースを1行変えて、もう一度ビルドしました。

```
rustnote:cache2 の大きさ: 170MB
/health: {"status":"ok"}
```

- 1回目（キャッシュが空）は、`cargo build` の段が 95.0 秒（依存クレートを含めて全部）でした。ソースを1行変えた2回目は **14.1 秒**で、コンパイルしたのは RustNote の1つだけ（`Compiling` の行が1行）でした。ビルド全体も 100 秒 → 24 秒です。
- できたイメージは 170MB で、`/health` も 200 でした。キャッシュのマウントは、イメージの中身を変えずに、作る時間だけを短くします。
- ただし、キャッシュのマウントは、ビルドする PC（や CI の機械）の中に残る物です。毎回まっさらな機械でビルドする CI では、効きません。その場合は、依存クレートだけを先にビルドする段を分ける（`cargo-chef` などの道具を使う）方法があります。

問3の解答:

- 例2-2の1段だけのイメージと同じく、root で動きます。`/app` の実行ファイルも、`/etc` の中も、RustNote のプロセスから書き換えられる状態になります。
- `USER app` は、RustNote の動きには何の影響も無い1行ですが、「万一のときに、被害を小さくする」ための大事な1行です。例2-1の `id` の結果（`uid=10001(app)`）で、毎回確かめられます。

まとめ

- RustNote の `Dockerfile` は、builder の段（`rust:1.99.1-trixie`・2.22GB）で release ビルドとアセットの束を作り、実行用の段（`debian:trixie-slim`）に、実行ファイル（17.1MB）と `assets/`（24.6kB）だけを運ぶ。できたイメージは 170MB で、利用者 `app` が動かし、書けるのは `/data` だけ。1段だけだと 3.98GB・root・道具入り。
- Docker のキャッシュは段ごと。`COPY . .` で材料が変わると、そのあとの `cargo build` は依存クレートから全部やり直し（101 秒）。時間がかかって変わらない手順は上に置き、`target/` はキャッシュのマウントで残せる。
- builder と実行用の Linux はそろえる。alpine（musl）では `exec … no such file or directory` で動かない。1つ古い Debian では今は動いたが、約束は無い。`COPY --from` の場所の書き間違いはビルドで止まり、`/data` を作り忘れると起動で止まる（SQLite の 14）。

この巻のまとめ

この版に収録した UNIT 01〜02 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: release ビルドと再現性

- ☐ debug と release の違いを、大きさ（約 21.7 MB と 11.8 MB）・速さ（ハッシュ化 約 400 ms と 20 ms）・整数のあふれ（panic と折り返し）で説明できる
- ☐ `cargo build --release --locked` → `topcoat asset bundle --release` の順で本番用の実行ファイルを作り、起動して `GET /health` で確かめられる

- ☐ 実行ファイルは「自分の隣の `assets\` 」を、DB は「今いるフォルダ」（または `DATABASE_URL` ）を使うことを説明できる
- ☐ `no asset bundle at ...` と `os error 10048` のエラーを読み、直し方を言える
- ☐ 同じソースから作った実行ファイルの中身が違う理由（リンクした時刻・識別番号）と、作る場所が中身に入り込むこと（ユーザーフォルダの場所・`asset!` のクレートの場所）を、実際の比べ方で説明できる
- ☐ release でも整数のあふれに気づけるよう、大きい型・`checked_*`・`overflow-checks = true` のどれかを選べる
- ☐ コミット・Rust の版・`Cargo.lock` と実行ファイルの指紋を、`release-record.txt` に記録できる

UNIT 02: multi-stage の Dockerfile

- ☐ RustNote の Dockerfile の builder の段と実行用の段が、それぞれ何を入れ、何を入れないかを、docker history の結果で説明できる
- ☐ 1段だけの Dockerfile と比べて、大きさ（3.98GB と 170MB）・中の道具・動かす利用者（root と app）の違いを言える
- ☐ Docker のキャッシュが段ごとに効くことと、ソースを1行変えると cargo build が全部やり直しになる理由を説明できる
- ☐ キャッシュのマウントで target/ を残し、やり直しを短くできる
- ☐ 実行用の段の Linux のちがい（alpine で no such file or directory）・COPY --from の場所・/data の持ち主のまちがいが、それぞれどこで、どう止まるかを言える

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03（アセットの束を本番用に作る）・UNIT 04（fly launch・fly.toml・ポート）・UNIT 05（secrets・設定・入れ替え）・UNIT 06（health check と起動の順番）・UNIT 07（migration を出す順番と release command）・UNIT 08（deploy・ログ・動作の確かめ・独自ドメイン）・UNIT 09（台数・地域・費用・Turso との距離）・UNIT 10（rollback・backup・障害からの復旧・更新）は準備中で、次の更新から順に追加します。それまでの間、公開の全体の流れは、RustNote の `README.md` の「公開する（Fly.io + Turso）の流れ」と、`Dockerfile`・`fly.toml` の先頭のコメントで確かめられます。

用語ミニ辞典

この巻（UNIT 01〜02）に出てきた用語です。

UNIT 01 の用語

用語	意味
debug ビルド	`cargo build` で作る試し用の実行ファイル。最適化なし、整数のあふれを確かめる。`target\debug` にできる
release ビルド	`cargo build --release` で作る本番用の実行ファイル。最適化あり、既定では整数のあふれを確かめない。`target\release` にできる
最適化	同じ動きのまま、速く小さくなるように、コンパイラーがプログラムを作り直すこと
`cfg!(debug_assertions)`	debug ビルドなら `true`、release ビルドなら `false` になる式
整数のあふれ（オーバーフロー）	計算の答えが、型で表せる範囲を超えること。debug は panic、release は既定で折り返す
`checked_add` / `checked_mul`	あふれたら `None` を返す計算。あふれたことをプログラムで扱える
`overflow-checks`	`[profile.release]` に書くと、release でもあふれを確かめる設定
アセットの束	`topcoat asset bundle` が作る、CSS などのファイルと目録（`manifest.toml`）。実行ファイルの隣の `assets\` に置く
再現性	同じソース・同じ道具・同じ部品から、いつ誰が作っても同じ物ができること
SHA-256 の指紋	ファイルの中身から計算した長い値。1 バイトでも違えば、まったく別の値になる。`Get-FileHash` で表示できる
`/Brepro`	Windows のリンカーに、時刻の代わりに中身から計算した値を使わせる指定
`--remap-path-prefix`	実行ファイルに入れるフォルダの場所の文字を、別の文字に置き換える rustc の指定
`RUSTFLAGS`	cargo が rustc に渡す追加の指定を書く環境変数。変えると全部のクレートが作り直しになる
os error 10048	Windows の「そのアドレスとポートは、もう使われている」エラー

UNIT 02 の用語

用語	意味
multi-stage ビルド	1つの Dockerfile に FROM を2つ以上書き、前の段で作った物から要る物だけを COPY --from で運ぶ作り方
レイヤー	イメージを作る1つ1つの手順の結果。同じ手順・同じ材料なら、前のビルドの結果（キャッシュ）を使い回す
`CACHED`	docker build の記録で、その手順の結果を前のビルドから使い回したという印
glibc	Linux の C の基本のライブラリ。新しい glibc で作った実行ファイルは、古い glibc の上では動かないことがある
`docker history`	イメージのレイヤーと、それぞれの大きさ・作った手順を並べて見るコマンド
キャッシュのマウント	`RUN --mount=type=cache,…` で、ビルドをまたいでフォルダ（target/ など）を取っておくしくみ。イメージには残らない
最小権限	プログラムに、要る分だけの権限しか与えない考え方。RustNote は app で動き、書けるのは /data だけ